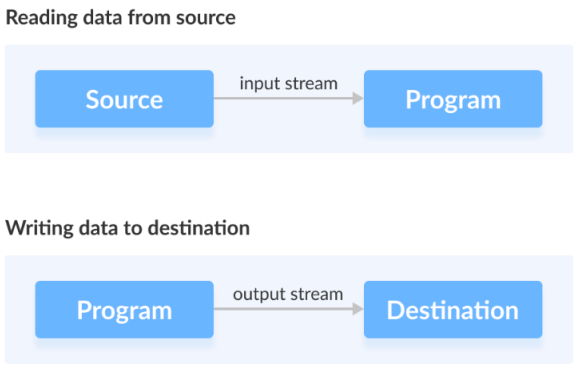
Streams

- In every programming language, we need to have a mechanism to read input data and also send the processed data that is also known as Output to the end-user.

- As you are aware, there are many input and output **devices** in the software world that programmers can use to read data from and write or output data to.

- Reading and writing data sources and destinations include:

    - Files
    - a keyboard
    - Network socket
    - In-memory Buffers (E.g: arrays)
    - Console
- Java programs perform I/O through **streams**.
- Streams are sequence of data that flow in one direction.
- Java implements streams within class hierarchies defined in the java.io package.



- There are two types of streams
    1. Input stream
    2. Output stream

An **input stream** is used to read data from the source. And, an **output stream** is used to write data to the destination.

Byte Stream Vs Character Stream

- There are two types of I/O streams in Java.
- One is **byte stream** and another one is **character stream**.
- **Byte stream** processes the data byte by byte i.e 8 bits at a time. **Character stream** processes the data character by character, processes the data of 16 bits at a time.

1. Byte Stream In Java :

- Byte stream reads and writes the data byte by byte i.e 8 bits maximum at a time.
- This stream is most suitable to process the binary files like image files, audio files, video files, executable files etc.
- All byte stream classes in Java are of type InputStream and OutputStream.
- FileInputStream, BufferedInputStream, DataInputStream, ObjectInputStream, ByteArrayInputStream are some byte stream classes in Java which are used for input i.e for reading data from source file.
- FileOutputStream, BufferedOutputStream, DataOutputStream, ObjectOutputS tream, ByteArrayOutputStream are some byte stream classes in Java which are used for output i.e for writing data to destination file.

- The subclasses of **byte stream- InputStream** class are given in the following table

Class	Description
BufferedInputStream	This class provides methods to read bytes from the buffer.
ByteArrayInputStream	This class provides methods to read bytes from the byte array.
DataInputStream	This class provides methods to read Java primitive data types.
FileInputStream	This class provides methods to read bytes from a file.
FilterInputStream	This class contains methods to read bytes from the other input streams, which are used as the primary source of data.
ObjectInputStream	This class provides methods to read objects.
PipedInputStream	This class provides methods to read from a piped output stream to which the piped input stream must be connected.
SequenceInputStream	This class provides methods to connect multiple Input Stream and read data from them.

- The subclasses of byte stream- OutputStream class are given in the following table

Class	Description
BufferedOutputStream	This class provides methods to write the bytes to the buffer.
ByteArrayOutputStream	This class provides methods to write bytes to the byte array.
DataOutputStream	This class provides methods to write the java primitive data types.
FileOutputStream	This class provides methods to write bytes to a file.
FilterOutputStream	This class provides methods to write to other output streams.
ObjectOutputStream	This class provides methods to write objects.
PipedOutputStream	It provides methods to write bytes to a piped output stream.
PrintStream	It provides methods to print Java primitive data types.

2. Character Stream In Java :

- Character stream reads and writes the data character by character.
- character streams in Java read and write data of **16 bits** maximum at a time.
- Character streams are mainly used to process text files.
- All character stream classes in Java are of type Reader and Writer.
- FileReader, BufferedReader, CharacterArrayReader, PipedReader, Input StreamReader are some character stream classes in Java which are used for input i.e for reading a file.
- FileWriter, BufferedWriter, CharacterArrayWriter, PipedWriter, InputSt reamWriter are some character stream classes in Java which are used for output i.e for writing a file.
- all names of character streams end with either Reader or Writer.

- The subclasses of the Reader class are given in the following table

Class	Description
BufferedReader	This class provides methods to read characters from the buffer.
CharArrayReader	This class provides methods to read characters from the char array.
FileReader	This class provides methods to read characters from the file.
FilterReader	This class provides methods to read characters from the underlying character input stream.
InputStreamReader	This class provides methods to convert bytes to characters.
PipedReader	This class provides methods to read characters from the connected piped output stream.
StringReader	This class provides methods to read characters from a string.

- The subclasses of the Writer class are given in the below table

Class	Description
BufferedWriter	This class provides methods to write characters to the buffer.
FileWriter	This class provides methods to write characters to the file.
CharArrayWriter	This class provides methods to write the characters to the character array.
OutpuStreamWriter	This class provides methods to convert from bytes to characters.
PipedWriter	This class provides methods to write the characters to the piped output stream.
StringWriter	This class provides methods to write the characters to the string.

Reading and Writing at Console in Java

1.Read input from console

- In Java, there are three different ways for reading input from the user in the command line environment(console).

    1. Using BufferedReader Class
    2. Using Scanner Class
    3. Using Console Class

- **Using the BufferedReader Class**

    - BufferedReader is a class in the java.io package in Java that provides buffering for efficient reading of characters from an input stream, such as reading from files, input streams, or console input.
    - reducing the number of I/O operations.
    - Using this technique we need to wrap **InputStreamReader** and **System.in** in the **BufferedReader** class.
    - This is done using the following **syntax**:
      BufferedReader br = new BufferedReader (new InputStreamReader (System.in));
    - BufferedReader Class is defined in java.io class so for using BufferedReader Class you need to import **java.io** first.

- **Using the Scanner Class**

    - Reading input data using the **Scanner** class is the most commonly used method. This way of the reading method is used by wrapping the **System.in** (standard input stream) which is wrapped in a **Scanner**, we can read input from the console.
    - Scanner class is defined in **java.util** so you need to import this first.
    - **syntax** is as follows:

- Scanner obj_name = new Scanner (System.in);
- methods Scanner class provides are as follows:

next()	It reads the next token as a String.
nextLine()	It reads the next input line as a String.
nextInt()	It reads the next input value as an integer.
nextDouble()	It reads the next input value as a double value.

- **next() Method:** This method can read the input only until a space(" ") is encountered. In other words, it finds and returns the next complete token from the scanner.
- **<u>nextLine() Method</u>:** This method can read the input till the end of line. In other words, it can take input until the line change or new line and ends input of getting '\n' or press enter.

3. Using the Console Class

- This is another way of reading user input from the console in Java.
- This technique also uses System.in for reading the input.
- This technique is best suited for reading input which does not require echoing of user input like reading user passwords. This technique **reads user inputs without echoing the characters entered by the user**.
- The Console class is defined in the **java.io** class which needs to be imported before using the console class.

Writing Console Output in Java

- Console output is most easily accomplished with print( ) and println( ).
- The PrintStream is a bult-in class that provides two methods print() and println() to write console output.
- Both print() and println() methods are used with System.out stream.
- The print() method writes console output in the same line.
- The println() method writes console output in a separate line (new line).

Reading and writing files

- The Java file class java.io.File gives you the access for
    - Creating files
    - Check if the file or directory exits or not
    - Read the length of the file
    - Rename a file
    - Delete a file
    - Move a file from one directory to another.
- Syntax to create a file
    - File file=new File(String fname)
- where fname can be a file name with absolute path.
- Java provides number of classes to read and write files.
- Two classes to be used to read and write files using byte streams are
    - FileOutputStream(String fname)
    - FileInputStream(String fname)
- When you are done with a file, you must close it.
- This is done by calling the close( ) method
- **FileOutputStream** class is used to create a file and write data into it.
- The stream would create a file, if it doesn't already exist, before opening it for output.
- To write data to output stream we use the method write() method. This method writes the specified byte to the output stream.